# Lab Report

## Subject: Cryptography

## Title: Implementation of Classical Cipher Techniques in C

---

## 1. Objective

The objective of this laboratory work is to demonstrate the working of various **classical cipher techniques** by implementing them using the **C programming language**. This lab helps in understanding basic encryption and decryption processes used in classical cryptography.

---

## 2. Theory

Cryptography is the science of securing information by transforming it into an unreadable format so that only authorized parties can access it. Classical cryptography mainly relies on **substitution** and **transposition** techniques. Although these methods are not secure by modern standards, they form the foundation of modern cryptographic algorithms.

---

## 3. Caesar Cipher

### 3.1 Description

The Caesar Cipher is one of the simplest and oldest substitution cipher techniques. In this method, each letter in the plaintext is shifted by a fixed number of positions down the alphabet. The number of positions shifted is known as the **key**.

For example, with a shift of 3: - A → D - B → E - Z → C

### 3.2 Algorithm

**Encryption:** 1. Take the plaintext message 2. Choose a shift value (key) 3. Shift each alphabetic character by the key value 4. Generate the ciphertext

**Decryption:** 1. Take the ciphertext 2. Shift each character backward by the same key 3. Obtain the original plaintext

### 3.3 Source Code (C)

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
   char plain_text[20], cipher_text[20];
   int key, i, length;

   printf("Enter the plain text: ");
   scanf("%s", plain_text);

   printf("Enter the key value: ");
   scanf("%d", &key);
```

```
  key = key % 26;   // To handle large key values
  length = strlen(plain_text);

  printf("\nThe plain text is: %s", plain_text);
  printf("\nThe encrypted text is: ");

  /* Encryption */
  for (i = 0; i < length; i++)
  {
    if (isupper(plain_text[i]))
    {
      cipher_text[i] = ((plain_text[i] - 'A' + key) %
26) + 'A';
    }
    else if (islower(plain_text[i]))
    {
      cipher_text[i] = ((plain_text[i] - 'a' + key) %
26) + 'a';
    }
    else
    {
      cipher_text[i] = plain_text[i];
    }

    printf("%c", cipher_text[i]);
  }
  printf("\nThe decrypted text is: ");

  /* Decryption */
  for (i = 0; i < length; i++)
  {
    if (isupper(cipher_text[i]))
    {
      plain_text[i] = ((cipher_text[i] - 'A' - key + 26)
% 26) + 'A';
    }
    else if (islower(cipher_text[i]))
    {
      plain_text[i] = ((cipher_text[i] - 'a' - key + 26)
% 26) + 'a';
    }
    else
    {
      plain_text[i] = cipher_text[i];
    }

    printf("%c", plain_text[i]);
  }

  return 0;
}
```

```
Enter the plain text: Mahesh
Enter the key value: 4

The plain text is: Mahesh
The encrypted text is: Qeliwl
The decrypted text is: Mahesh
```

### 3.4 Result

The plaintext is successfully converted into ciphertext using the shift key, and the original message is recovered after decryption.

---

# 4. Monoalphabetic Cipher

## 4.1 Description

A Monoalphabetic Cipher is a substitution cipher where each letter of the plaintext is replaced with a corresponding letter from a fixed substitution alphabet. Unlike the Caesar Cipher, the substitution is not limited to shifting and can be any random mapping of alphabets.

## 4.2 Algorithm

**Encryption:** 1. Define a substitution key (mapping of alphabets) 2. Replace each plaintext character with its mapped character 3. Generate the ciphertext

**Decryption:** 1. Use the reverse mapping of the substitution key 2. Replace each ciphertext character accordingly 3. Retrieve the original plaintext

## 4.3 Source Code (C)

```
#include <stdio.h>
#include <ctype.h>

char mono_encrypt(char);

/* Monoalphabetic substitution table */
char alpha[26][2] = {
  {'a','f'}, {'b','a'}, {'c','g'}, {'d','u'}, {'e','n'},
  {'f','i'}, {'g','j'}, {'h','k'}, {'i','l'}, {'j','m'},
  {'k','o'}, {'l','p'}, {'m','q'}, {'n','r'}, {'o','s'},
  {'p','t'}, {'q','v'}, {'r','w'}, {'s','x'}, {'t','y'},
  {'u','z'}, {'v','b'}, {'w','c'}, {'x','d'}, {'y','e'},
  {'z','h'}
};

int main()
{
  char str[50], cipher[50];
  int i;

  printf("Enter the string: ");
  fgets(str, sizeof(str), stdin);

  /* Encryption */
  for (i = 0; str[i] != '\0'; i++)
  {
    cipher[i] = mono_encrypt(str[i]);
  }
  cipher[i] = '\0';

  printf("\nPlain Text  : %s", str);
  printf("Cipher Text  : %s", cipher);

  return 0;
}

/* Function to encrypt a character */
char mono_encrypt(char ch)
{
  int i;

  if (isalpha(ch))
  {
    ch = tolower(ch);

    for (i = 0; i < 26; i++)
    {
      if (ch == alpha[i][0])
        return alpha[i][1];
    }
  }

  /* Return character unchanged if not alphabet
*/
  return ch;
}
```

```
Enter the string: Mahesh

Plain Text   : Mahesh
Cipher Text  : qfknxk
```

### 4.4 Result

The plaintext is encrypted using a fixed substitution alphabet and successfully decrypted using the reverse mapping.

---

## 5. Hill Cipher

### 5.1 Description

The Hill Cipher is a polygraphic substitution cipher based on linear algebra. It uses matrix multiplication modulo 26 to encrypt blocks of plaintext letters. The key is a square matrix, and its inverse is used for decryption.

### 5.2 Algorithm

**Encryption:** 1. Convert plaintext characters into numerical values (A=0 to Z=25) 2. Arrange plaintext into vectors of size n 3. Multiply the vector with the key matrix modulo 26 4. Convert the result back to characters

**Decryption:** 1. Find the inverse of the key matrix modulo 26 2. Multiply ciphertext vectors with the inverse matrix 3. Convert numbers back to characters

### 5.3 Source Code (C)

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int mod26(int x);

/* 2x2 Key Matrix (must have determinant
coprime with 26) */
int key[2][2] = {
  {3, 3},
  {2, 5}
};

int main()
{
  char plain[50], cipher[50];
  int i, p1, p2, c1, c2;

  printf("Enter the plain text: ");
  scanf("%s", plain);

  int len = strlen(plain);

  /* If length is odd, append 'x' */
  if (len % 2 != 0)
  {
    plain[len] = 'x';
    plain[len + 1] = '\0';
    len++;
  }

  /* Encryption */
  for (i = 0; i < len; i += 2)
  {
    p1 = tolower(plain[i]) - 'a';
    p2 = tolower(plain[i + 1]) - 'a';

    c1 = mod26(key[0][0] * p1 + key[0][1] * p2);
```

```
    c2 = mod26(key[1][0] * p1 + key[1][1] * p2);

    cipher[i]     = c1 + 'a';
    cipher[i + 1] = c2 + 'a';
  }

  cipher[len] = '\0';

  printf("\nPlain Text  : %s", plain);
  printf("\nCipher Text : %s\n", cipher);
```

```
Enter the plain text: Mahesh

Plain Text  : Mahesh
Cipher Text : kyhixt
```

### 5.4 Result

The plaintext is encrypted using matrix operations and correctly decrypted using the inverse matrix.

---

## 6. Vigenère Cipher

### 6.1 Description

The Vigenère Cipher is a polyalphabetic substitution cipher that uses a keyword to encrypt the plaintext. Each letter of the plaintext is shifted according to the corresponding letter of the keyword.

### 6.2 Algorithm

**Encryption:** 1. Choose a keyword 2. Repeat the keyword to match the plaintext length 3. Shift each plaintext character based on the keyword character 4. Produce the ciphertext

**Decryption:** 1. Use the same keyword 2. Shift characters backward based on keyword values 3. Obtain the original plaintext

### 6.3 Source Code (C)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Function prototypes */
void encryptVigenere(char *, char *, char *);
void decryptVigenere(char *, char *, char *);

int main()
{
  char plaintext[100], key[100], ciphertext[100],
decrypted[100];

  printf("Enter the plaintext: ");
  fgets(plaintext, sizeof(plaintext), stdin);

  /* Remove newline if fgets reads it */
  plaintext[strcspn(plaintext, "\n")] = '\0';

  printf("Enter the key: ");
  fgets(key, sizeof(key), stdin);
  key[strcspn(key, "\n")] = '\0';

  encryptVigenere(plaintext, key, ciphertext);
  return 0;
}

/* Function to handle modulo 26 */
int mod26(int x)
{
  return (x % 26 + 26) % 26;
}
```

```
    printf("\nEncrypted Text: %s\n", ciphertext);

    decryptVigenere(ciphertext, key, decrypted);
    printf("Decrypted Text: %s\n", decrypted);

    return 0;
}

/* Encryption function */
void encryptVigenere(char *text, char *key, char
*cipher)
{
    int i, j = 0;
    int keyLen = strlen(key);

    for (i = 0; text[i] != '\0'; i++)
    {
        char ch = text[i];

        if (isalpha(ch))
        {
            int k = tolower(key[j % keyLen]) - 'a';  // key
index
            if (isupper(ch))
                cipher[i] = ((ch - 'A' + k) % 26) + 'A';
            else
                cipher[i] = ((ch - 'a' + k) % 26) + 'a';

            j++;  // increment key index only for letters
        }
        else
        {
            cipher[i] = ch;  // non-letter characters
remain unchanged
        }
    }
    cipher[i] = '\0';
}

/* Decryption function */
void decryptVigenere(char *cipher, char *key,
char *text)
{
    int i, j = 0;
    int keyLen = strlen(key);

    for (i = 0; cipher[i] != '\0'; i++)
    {
        char ch = cipher[i];

        if (isalpha(ch))
        {
            int k = tolower(key[j % keyLen]) - 'a';  // key
index
            if (isupper(ch))
                text[i] = ((ch - 'A' - k + 26) % 26) + 'A';
            else
                text[i] = ((ch - 'a' - k + 26) % 26) + 'a';

            j++;  // increment key index only for letters
        }
        else
        {
            text[i] = ch;  // non-letter characters remain
unchanged
        }
    }
    text[i] = '\0';
}
```

```
Enter the plaintext: Mahesh
Enter the key: apple

Encrypted Text: Mpwpwh
Decrypted Text: Mahesh
```

## 6.4 Result

The plaintext is securely encrypted using a keyword and successfully decrypted using the same key.

# 7. Rail Fence Cipher

## 7.1 Description

The Rail Fence Cipher is a transposition cipher that writes the plaintext in a zig-zag pattern across multiple rails and then reads it row by row to form the ciphertext.

## 7.2 Algorithm

**Encryption:** 1. Choose the number of rails 2. Write the plaintext in a zig-zag manner across rails 3. Read the characters row-wise to form ciphertext

**Decryption:** 1. Reconstruct the zig-zag pattern 2. Fill characters row-wise 3. Read the message in zig-zag order

## 7.3 Source Code (C)

```
#include <stdio.h>
#include <string.h>

/* Function prototypes */
void encryptRailFence(char *, int, char *);
void decryptRailFence(char *, int, char *);

int main() {
  char plaintext[100], ciphertext[100],
decrypted[100];
  int key;

  printf("Enter the plaintext: ");
  fgets(plaintext, sizeof(plaintext), stdin);
  plaintext[strcspn(plaintext, "\n")] = '\0'; //
remove newline

  printf("Enter the key (number of rails): ");
  scanf("%d", &key);

  encryptRailFence(plaintext, key, ciphertext);
  printf("\nEncrypted Text: %s\n", ciphertext);

  decryptRailFence(ciphertext, key, decrypted);
  printf("Decrypted Text: %s\n", decrypted);

  return 0;
}

/* Encryption function */
void encryptRailFence(char *text, int key, char
*cipher) {
  int len = strlen(text);
  int i, row, col = 0;
  int dir_down;

  /* Create the rail matrix */
  char rail[key][len];
  for (i = 0; i < key; i++)
    for (int j = 0; j < len; j++)
      rail[i][j] = '\n';

  row = 0;
  dir_down = 0; // initially moving up

  for (i = 0; i < len; i++) {
    rail[row][col++] = text[i];

    if (row == 0 || row == key - 1)
      dir_down = !dir_down;

    row += dir_down ? 1 : -1;
  }

  /* Read the matrix row-wise to get cipher text
*/
  int idx = 0;
  for (i = 0; i < key; i++)
    for (int j = 0; j < len; j++)
      if (rail[i][j] != '\n')
        cipher[idx++] = rail[i][j];

  cipher[idx] = '\0';
}

/* Decryption function */
void decryptRailFence(char *cipher, int key, char
*text) {
```

```
    int len = strlen(cipher);
    int i, row, col;
    int dir_down;

    /* Create the rail matrix */
    char rail[key][len];
    for (i = 0; i < key; i++)
        for (int j = 0; j < len; j++)
            rail[i][j] = '\n';

    /* Mark the positions with '*' */
    row = 0;
    dir_down = 0;
    for (i = 0; i < len; i++) {
        rail[row][i] = '*';

        if (row == 0 || row == key - 1)
            dir_down = !dir_down;

        row += dir_down ? 1 : -1;
    }

    /* Fill the cipher text in marked positions */
    int idx = 0;
    for (i = 0; i < key; i++)
        for (int j = 0; j < len; j++)
            if (rail[i][j] == '*' && idx < len)
                rail[i][j] = cipher[idx++];

    /* Read the matrix in zig-zag to get plaintext */
    row = 0;
    dir_down = 0;
    idx = 0;
    for (col = 0; col < len; col++) {
        text[idx++] = rail[row][col];

        if (row == 0 || row == key - 1)
            dir_down = !dir_down;

        row += dir_down ? 1 : -1;
    }

    text[idx] = '\0';
}
```

```
Enter the plaintext: Mahesh
Enter the key (number of rails): 3

Encrypted Text: Msaehh
Decrypted Text: Mahesh
```

### 7.4 Result

The plaintext is rearranged using a rail-based pattern and correctly restored after decryption.

---

## 8. Conclusion

In this laboratory work, various classical cipher techniques were successfully studied and implemented using the C programming language. Each cipher demonstrated a different approach to encryption, ranging from simple substitution to matrix-based transformations and transposition techniques. Although these ciphers are not secure against modern attacks, they play a crucial role in understanding the fundamentals of cryptography and form the basis for advanced encryption algorithms.

---

## 9. References


1. William Stallings, *Cryptography and Network Security*

2. Cryptography lecture notes
3. Standard textbooks on Information Security